ACS Recognition of Prior Learning (RPL) Form

COMPLETING THE FORM

As part of ACS's Recognition of Prior Learning application pathway, applicants must:

1. Fill and complete the **Recognition of Prior Learning (RPL) Form** in its entirety.
2. Convert the completed RPL Form into a PDF document (i.e., "Save as Adobe PDF"). **Note:** Scanned versions of the form will not be accepted.
3. Save the file including your Application Number and Name in the file name (e.g., "ACS-0312235 Jane Doe RPL").
4. Upload the Form PDF from Step 3 into the ACS Skills Assessment Application.

Applicants must complete the following sections in their entirety:

1. **Applicant Details & Ethical Conduct**
2. **Non-IT Tertiary Qualifications**
3. **Professionalism and Body of Knowledge**
4. **RPL Project Reports (two required)**

1. APPLICANT DETAILS & ETHICAL CONDUCT

Applicant Full Name	**MELVIN MALABANAN**
Applicant Email Address	**MMALABAN@HOTMAIL.COM**
Applicant Date of Birth	30/12/1983
LinkedIn Profile URL (if any)	HTTPS://WWW.LINKEDIN.COM/IN/MMALABAN/
Professional Portfolio URL (if any)	

In lodging my RPL Form, I acknowledge that I have read and understood the **ACS MSA Service Agreement**. I understand that I must abide by the **ACS Code of Professional Conduct** and the **ACS Code of Professional Ethics**.

I confirm that the contents of this form are my original submission and no part of it has been copied from any other source except where due acknowledgement is made. I confirm that the submitted explanation of my knowledge and projects truthfully and accurately describe me and my personal involvement in the projects. Furthermore, this submission has not been previously submitted for this or any other application.

In submitting this form as part of an ACS Migration Skills Assessment application, I give ACS permission to retain, compare, copy, and archive for the purposes of detecting plagiarism and/or review by external assessors. I am aware that plagiarism or misrepresentation will automatically invalidate this application, will jeopardise any future applications, and may be reported to the Australian Department of Home Affairs.

2. NON-IT TERTIARY QUALIFICATIONS

Please complete the following information regarding your non-IT Tertiary Qualification/s.

Have you completed a non-IT tertiary qualification (e.g., bachelor's degree or higher)?	☒**Yes** ☐**No**

If you selected "Yes," provide the details for up to two **non-IT tertiary qualifications** below:

Non-IT Qualification #1

Name of Degree or Award	**BACHELOR OF SCIENCE IN ELECTRICAL ENGINEERING**
University or Institution Name	**UNIVERSITY OF MICHIGAN - DEARBORN**
Country of Study	USA
Normal Duration (years/semesters)	4 YEARS
Completion Date	17/12/11

Non-IT Qualification #2

Name of Degree or Award	**ASSOCIATE DEGREE, ENGINEERING**
University or Institution Name	**SCHOOLCRAFT COLLEGE**
Country of Study	USA
Normal Duration (years/semesters)	2 YEARS
Completion Date	26/04/08

Note: The above qualifications will not be assessed by ACS. An assessor may only use these qualifications as a reference point to support your professional history, skill development, and career progression when reviewing your application.

3. PROFESSIONALISM AND BODY OF KNOWLEDGE

Section 3 will be assessed based on the following document: The ICT Profession, Professionals and the ACS Body of ICT Knowledge (CBoK). Please ensure that you have read and understood the "CBoK."

Hallmarks of an ICT Professional and their Topic Areas

To complete this section, you must select two topic areas (lettered) from each hallmark below. A total of four topics will be discussed in this section.

Professionalism as it applies in ICT

a. Professional ICT Ethics
b. Impact of ICT
c. Working Individually and in ICT development teams
d. Professional Communication
e. The Professional ICT Practitioner

Core ICT Knowledge

a. ICT Fundamentals
b. ICT Infrastructure
c. Information and Data Science Engineering
d. Computational Science and Engineering
e. Application Systems
f. Cyber Security
g. ICT Projects
h. ICT Management and Governance

Professionalism as it Applies in ICT or Data Science

Name your selected two topic areas from the list above (a-e).

Topic Area 1	Working Individually and in ICT Development Teams
Topic Area 2	Professional Communication

How have you developed your skills and knowledge during your professional experience for each of the above?

- Illustrate your depth of knowledge providing clear and concise explanation.
- Limit your response to no more than one and a half pages (approx. 750 words max)

My professional experience has developed these two professionalism topics through independent technical ownership, collaboration in multidisciplinary ICT teams, and communication with technical, managerial, and customer-facing audiences. In the automotive software domain, I have repeatedly had to work effectively within an organizational context while also contributing to broader team objectives. This has strengthened my ability to manage my own work, collaborate with diverse teams, and communicate clearly across different teams or audiences.

I first developed these skills strongly at Vector, where I worked closely with OEM and supplier engineers while providing embedded software support, integration assistance, and customer training. In that environment, I often moved between individual technical problem-solving and collaborative issue resolution. I might investigate a software integration issue independently, but successful delivery still depended on working with product developers, test engineers, and customer teams. This developed my ability to operate effectively in both individual and group settings, and to understand how cooperation, role clarity, and responsiveness affect engineering outcomes.

I strengthened my ability to work independently further at BorgWarner by taking responsibility for technically complex embedded software tasks and delivering them with minimal supervision. I designed and implemented software modules within AUTOSAR-based diagnostic and electrification platforms, which required me to manage my own workflow, interpret requirements, plan implementation tasks, and validate outcomes against technical and project constraints. A clear example was the development of the InfraDID Config automation tool, where I worked as the sole software designer and developer. I identified an inefficient manual process, defined the technical approach, implemented the Python-based solution, and validated the generated outputs for

integration into the base software. This experience strengthened my ability to manage time, structure technical work, and deliver outcomes independently while remaining aligned with the engineering needs of the organization.

I then expanded these teamwork capabilities further at IAV, where I worked within multidisciplinary development teams made up of software engineers, system engineers, software architects, testers, project managers, and customer-facing stakeholders. I led software requirement reviews for controller modules such as battery management systems, gateway modules, and memory seat modules. These activities required me to contribute my own technical judgment while also incorporating feedback from other disciplines, aligning software decisions with design constraints, and supporting shared delivery goals. Through this progression, I learned that effective teamwork in ICT depends not only on technical ability, but also on coordination, responsiveness, and respect for the roles of other contributors.

Professional communication has also developed progressively across these roles. At Vector, I built the foundation through customer support, technical discussions, and formal training delivery. I delivered customer training and courseware for embedded software systems, which required me to explain concepts clearly in meetings, workshops, and classroom-style sessions. This taught me how to adjust technical language for audiences with different levels of familiarity and different project responsibilities.

At BorgWarner and later at IAV, I strengthened this further through more complex requirements discussions, design reviews, and technical documentation. In embedded software projects, communication has to be accurate, structured, and appropriate for the audience. I regularly communicated with system engineers, software architects, managers, and customers to clarify requirements, discuss design feasibility, explain implementation trade-offs, and document expected behavior and acceptance criteria. I have also written and reviewed software requirements, design notes, implementation documentation, and test-related material in ASPICE-oriented environments. At IAV, I expanded this capability further through my AI Mentor responsibilities, where I led workshops, produced guides and FAQs, and coached colleagues on the practical use of AI tools. These experiences strengthened my ability to communicate in different forums, adapt style to suit the audience, and support adoption of technical practices through clear explanation.

Across these roles, I have learned that professionalism in ICT is demonstrated through disciplined individual work, effective collaboration in development teams, and clear communication that supports sound decision-making. Working individually requires ownership, workflow discipline, and accountability. Working in ICT teams requires collaboration, coordination, and the ability to contribute effectively within an organizational structure. Professional communication requires adapting technical content for different audiences and documenting work in forms that support implementation, review, and ongoing maintenance. My experience in embedded automotive software has consistently developed these capabilities, and they now form a core part of how I approach professional ICT practice.

Core ICT or Data Science Knowledge

Name your selected two topic areas from the list above (a-h).

Topic Area 3	ICT Infrastructure
Topic Area 4	Application Systems

How have you developed your skills and knowledge during your professional experience for each of the above?

- Illustrate your depth of knowledge providing clear and concise explanation.
- Limit your response to no more than one and a half pages (approx. 750 words max)

My professional experience has developed these two core ICT knowledge areas through work on embedded automotive platforms, AUTOSAR-based software, distributed controller architectures, and system integration projects across Vector, BorgWarner, and IAV.

I developed my knowledge of ICT Infrastructure first at Vector, where I worked on embedded software solutions based on CANbedded. This stack provided support for the CAN driver, diagnostics, and communication layers, including higher-layer abstraction to simplify integration into customer application software. Most of these customers were Tier 1 suppliers for Ford. This work gave me practical exposure to embedded hardware environments, layered communication software, and the role of software within networked vehicle systems.

At BorgWarner, this knowledge became deeper through AUTOSAR-based platform work. My first BorgWarner project was the Transfer Case Control Module (TCCM) used in GM vehicles. This module manages 4WD shifting operations by processing driver shift requests, controlling the transfer case encoder motor, and verifying shift completion. It was the first BorgWarner project to use AUTOSAR, and I worked as the main software integrator. In one release, I also created the first version of the device driver for a then-new Infineon System Basis Chip (SBC). My initial implementation controlled the power rails, CAN transceiver, and watchdog behavior within the SBC. This was the first time I developed a device driver from scratch for a product delivery, which strengthened my ability to convert hardware behavior and product requirements into production-ready low-level software within an AUTOSAR-based embedded platform.

I expanded this infrastructure knowledge further through the TSGen project, which focused on a 48V hybrid electric vehicle powertrain system for Audi. The system integrated an electric motor, transmission, and inverter into a single unit and communicated using FlexRay. This gave me broader exposure to powertrain-oriented embedded infrastructure, communication protocols beyond CAN and LIN, and integration challenges in electrically and mechanically coupled systems. It also influenced how I later approached diagnostic architecture and inspired the development of the InfraDID Config tool for standardizing infrastructure-level diagnostic data implementation.

At IAV, I expanded my infrastructure knowledge into distributed and cyber-physical systems. I designed and implemented layered embedded software architecture for the NXP S32K platform using an internal framework, and I worked on a multi-processor Software-Defined Vehicle (SDV) platform using NVIDIA Jetson for cloud connectivity and GUI, Renesas R-Car for application execution, and the IAV Dragoon ECU for vehicle interfacing. This strengthened my understanding of distributed systems, system partitioning, communication links between processing environments, and the relationship between low-level software, communication services, and application logic.

I developed my knowledge of Application Systems through roles that required me to understand how software components support operational goals, user expectations, and system behavior in real contexts. At Vector, I supported embedded software solutions built around the CANbedded stack. Because these projects were intended for

Tier 1 suppliers serving Ford programs, I had to understand how communication and diagnostic layers would be integrated into each customer's application software and adapted to their system constraints. This gave me early exposure to the context in which application software is deployed and used.

At BorgWarner, I strengthened this area through projects where software behavior was directly tied to vehicle function and product goals. In the TCCM project, the application system had to respond correctly to driver shift requests, coordinate the logic needed to execute the requested 4WD mode change, and verify that the shift operation had completed successfully. In the TSGen project, software behavior had to support a 48V hybrid powertrain system integrating motor, transmission, and inverter functions. These projects strengthened my understanding of how application systems must coordinate multiple components into a coherent functional system that satisfies operational requirements, communication needs, and system-level expectations.

At IAV, I expanded this knowledge further through requirements engineering, software architecture, and system integration work. I translated customer requirements into structured software requirements using tools such as DOORS and Polarion, ensuring that the resulting design, implementation, and testing aligned with system objectives. The SDV demonstration platform is a strong example because it combined cloud connectivity, graphical user interaction, application execution, and vehicle interfacing into one coherent platform. This required analysis of how system components should interact, what responsibilities belonged to each processing environment, and how the resulting system would be understood and evaluated by customers.

Together, these experiences have given me an understanding of both ICT Infrastructure and Application Systems. I understand the hardware, communication protocols, systems software, and distributed environments that support embedded computing systems, and how software components must be integrated into coherent systems that satisfy technical and operational objectives.

4. RPL PROJECT REPORTS

In Section 4, you must demonstrate your command and implementation of the knowledge and skill development described in Section 3 (above) by providing **two project reports.**

Each Project Report must demonstrate your in-depth ICT or data science knowledge by:

1. Providing a clear written description of a significant completed project, engagement, or work episode undertaken by you during your professional career,
2. Highlighting your individual skills and contribution to a particular role/s in the profession
3. Confirming your professional currency: Project 1 must have been undertaken within the last two years, and Project 2 must have been within the last four years.

Selecting and Building Your Project Reports

1. A project longer than two years may be used for both reports if it meets at least one of the conditions below:
    a. The project has clearly defined work efforts which took place in parallel, each with their own solution development and design activities and their own deliverables.
    b. The project has clearly defined phases that were executed in succession, each with its own solution development and design activities and deliverables. A second project phase that constructs and implements the solution developed by the first phase does not meet this requirement.
2. If you have held non-project roles, such as ICT support or system administration, it is imperative that you supply details of your work experience and challenges faced during your tenure. Highlight your technical expertise, professional experience, contributions to process improvement, client interactions, and problem-solving.
3. Depending on the nature of your role in each project, the Project Report should cover an appropriate selection of factors based on the type of ICT project selected. Factors may include:
    a. System Analysis and Design and Software Engineering methodologies used.
    b. Contribution to the processes involved in the design and implementation of enterprise-wide computing systems.
    c. Programming languages, design paradigms and implementation procedures adopted.
    d. Database and/or file design and management techniques employed.
    e. Network topologies, including size, distribution and security facilities installed.
    f. Project Management and quality assurance techniques followed.
    g. Internet application design, including database interactivity and security measures implemented.
    h. Cybersecurity protocols for enhanced project security.
    i. Data science methodologies to optimize data analysis and decision-making processes.
    j. ICT managerial activities, demonstrating the nature and extent of responsibilities.

Project Summary

Project 1 Name	SDV Demonstrator
Start Date*	07/25
End Date	01/26

*This project must have been undertaken within the last two years.

Project 2 Name	InfraDID Config
Start Date^	02/22
End Date	04/22

^This project must have been undertaken within the last four years.

Project Report 1

Completing Your Project Report

- Reports should be written in first-person narrative (I, my, me) to provide clear discussion and ownership of what you did versus what others did. Provide your own thoughts. Do not copy project documentation.
- You must provide clear citation and referencing when drawing on the work of others. Other people's original ideas and methods should be clearly distinguished, and other people's words, illustrations and diagrams should be clearly indicated regardless of whether they are copied exactly, paraphrased, or adapted.
- Diagrams from the project documentation may be helpful, but the text should be in your own words. Please ensure that diagrams are relevant, readable, and help the assessor to understand what you did as a member of the project team.
- If sections or components of the Project Report (below) are not relevant to your participation in the project, then leave the section blank.

Focus on quality, rather than quantity. Each Project Report should be no more than 4-5 pages.

1. About Project 1

Project 1 Name	SDV Demonstrator			
Nature of Project	Multi-processor embedded software architecture and system integration project for a Software-Defined Vehicle demonstration platform			
Location/s of Project	Novi, Michigan, USA			
Name of Your Employer	IAV Automotive Engineering, Inc.			
Total Project Duration	From	07/25	To	01/26
Your Involvement*	From	07/25	To	01/26

*This project must have been undertaken within the last <u>two</u> years.

1.1 Client Details

Client's Business Name	IAV Automotive Engineering, Inc.	
Business Address	39525 MacKenzie Dr. Suite 1000 Novi, Michigan, 48377 USA	
Web Address	www.iav.com/us	
General Email Address	info@iav.com	
Contact Name/s	Chris Coyne	
Contact Phone Number/s	+1 (877) 926 - 8428	

1.2 Project Resources

Your Team Size	3

Total Project Team Size	5

1.3 Personal Involvement

a) List the project phases you were involved in.

Start	End	Phase Description
07/25	07/25	Requirements and concept definition
07/25	08/25	Software architecture design and platform partitioning
08/25	08/25	Integration of processing environments and communication interfaces
08/25	09/25	Demonstration validation and customer presentation support - for the first showcase (Ford)
09/25	10/25	Update for showcasing to another customer (Stellantis)
12/25	01/26	Update for showcasing to another customer using Lego Raptor and RC Car (GM)

b) Describe your role/s and responsibilities in the project.

I worked as a Senior Software Engineer on a Software-Defined Vehicle (SDV) demonstration platform at IAV. The project centered on a demonstration board containing NVIDIA Jetson, Renesas R-Car, and the IAV Dragoon ECU. Its purpose was to showcase IAV's SDV knowledge, architecture strategy, and deployment approach through a working embedded system that could be reused across different vehicles with minimal software change for customer-facing demonstrations. My primary responsibility was to lead the software architecture and integration of this multi-processor platform. I was responsible for defining how the major software and hardware components would interact, how responsibilities would be partitioned across the processing environments, and which technologies would be used to support the architecture, including Eclipse eCAL for communication and Docker-based containerization on the R-Car. After the design was established, I worked primarily on the R-Car side while another team member worked on the Jetson side. We jointly verified that communication between Jetson and R-Car was robust and consistent by sending large data sets in both directions, timing the transmission, and confirming stable behavior under loaded or stressed conditions appropriate for a demonstration platform. I also helped design how the Dragoon ECU software should behave so it could remain flexible across the Volkswagen ID.4, Ford Escape, Lego Raptor, and RC car setups, and I designed and implemented the build configuration for those different environments. A third team member implemented the signal-handling logic on the Dragoon based on that design. Because the Dragoon used a bootloader, updates could be performed efficiently over CAN. I also helped adapt the demonstration strategy when customer constraints changed, including a pivot from full vehicle demonstrations to smaller physical demonstrators that preserved the same architectural concept.

2. Business Opportunity or Problem

a) Describe the business opportunity or problem(s) this project addressed.

The project addressed the need to demonstrate a scalable Software-Defined Vehicle architecture that could support multiple hardware platforms, multiple vehicles, and multiple customer scenarios. The challenge was not only to build a working prototype, but to show

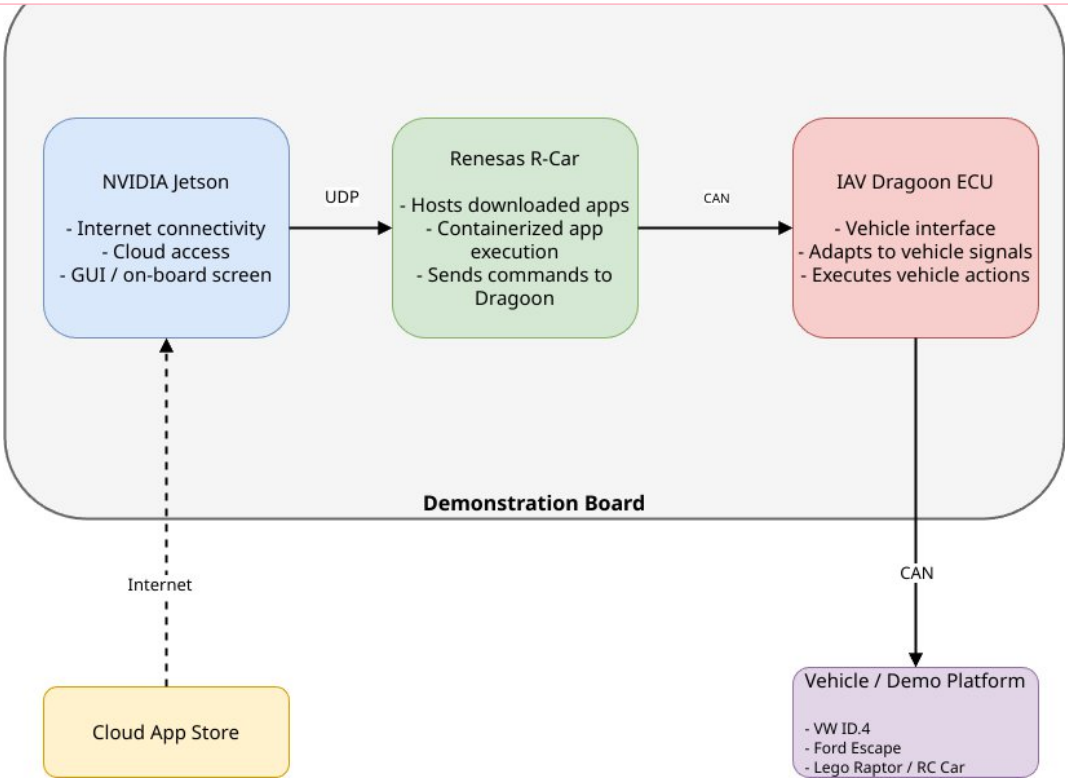
that a distributed embedded software architecture could separate responsibilities across different computing environments while still operating as a coherent system. The demonstration board had to show that one core platform could be installed into very different vehicles, in this case a 2021 Volkswagen ID.4 and a 2025 Ford Escape, with minimal software changes. The main software reflash was required only on the IAV Dragoon ECU because it communicated directly with each vehicle and therefore had to adapt to different signals, network topologies, and vehicle data. A further challenge emerged when GM requested a demonstration but did not allow vehicles inside its facility due to safety concerns. This required a pivot in the demonstration method while preserving the same SDV architecture and value proposition. The broader opportunity was to demonstrate how SDV concepts such as downloadable applications, modular deployment, and separated processing responsibilities could be applied in a practical automotive environment, even when the final showcase format had to change.

3. Solution

a) Discuss your contribution to the solution, project, or engagement.

I led the software architecture and integration of a multi-processor SDV demonstration platform using NVIDIA Jetson for internet connectivity and graphical user interface functions, Renesas R-Car for application hosting and execution, and the IAV Dragoon ECU for direct vehicle interfacing. The demonstration included an app-store concept connected to a cloud-based store capable of hosting different vehicle applications, ranging from convenience features to potential performance-related functions. My contribution focused on defining the software structure, assigning responsibilities to the different processing environments, and integrating the system components so that they operated as one platform rather than isolated devices. I selected Eclipse eCAL as the higher-level communication framework and Docker-based containerization on the R-Car so that downloaded applications could be isolated from the rest of the platform. Jetson to R-Car communication used eCAL over UDP, while R-Car to Dragoon used eCAL over SocketCAN so that SocketCAN handled the low-level CAN interface and eCAL provided the higher-level communication layer. I worked primarily on the R-Car implementation and integration, while coordinating with the Jetson implementation so the communication path remained robust during demonstrations. The integrated system then used the Dragoon ECU to command simple vehicle actions, such as blinking headlights or turn indicators in a defined pattern. When the GM demonstration constraint prevented the use of full vehicles, I helped pivot the showcase to a Lego Raptor and an RC car equipped with multiple LEDs to simulate headlights, brake lights, and indicator lights. This allowed us to preserve the same architectural demonstration while removing the dependency on full production vehicles.

For a better visualization, below is a block diagram of the system.



b) List the major deliverables of the project that you were responsible for or contributed to.

- Multi-processor SDV software architecture definition
- Integration approach for NVIDIA Jetson, Renesas R-Car, and IAV Dragoon ECU
- Communication architecture using Eclipse eCAL across the platform, with eCAL over UDP between Jetson and R-Car, and eCAL over SocketCAN between R-Car and Dragoon
- Software partitioning across cloud (design), application, and vehicle-interface responsibilities
- Integrated embedded software demonstrator for customer-facing scenarios
- Adapted demonstration platform using Lego Raptor and RC car with LED-based vehicle signal simulation

c) Describe any design or problem-solving methods you used on this project.

I approached the work by treating the platform as a distributed embedded system with clearly separated functional responsibilities. I first identified the major system roles required for the demonstration, including internet connectivity, graphical user interaction, application execution, and vehicle interfacing. I then mapped those responsibilities to the most appropriate processing environments and defined how they should communicate as part of a unified architecture. Communication across the platform leveraged Eclipse eCAL as the higher-level communication framework. The Jetson and R-Car communicated using eCAL over UDP while connected through a router, and the R-Car communicated with the Dragoon ECU using eCAL over SocketCAN, where SocketCAN handled the low-level CAN interface. From the Dragoon ECU, a wire harness carrying power and CAN connected the demonstrator into the vehicle. My design approach focused on modularity, partitioning, scalability, and practical software integration so that the demonstrator could be reused across different vehicles with minimal software change. I also used

containerization on the R-Car to separate application responsibilities and reduce the risk that one application could affect the rest of the platform. To validate the communication path, we exchanged large data sets between Jetson and R-Car, measured transmission timing, and confirmed stable operation under loaded conditions suitable for demonstration use. When the GM facility restriction prevented the use of actual vehicles, I applied the same architectural principles to a Lego Raptor and an RC car outfitted with LEDs to simulate vehicle lighting behavior. Iterative integration and validation were then used to confirm that both the full-vehicle and reduced demonstrator versions supported the same architectural message and demonstration objectives.

d) Was your solution/s implemented? If so, describe the role you had, if any, in the implementation.

Yes. The solution was implemented as a working SDV demonstration platform. I was directly involved in the software architecture and integration activities required to move the platform from concept to a demonstrable system. My role included defining the architecture, supporting the integration of the key processing environments, and helping validate that the platform behaved as intended during demonstrations in multiple vehicle platforms as well as in the adapted Lego Raptor and RC car demonstrators.

4. Results

a) Assess the overall success or failure of the project.

The project successfully demonstrated a scalable software-defined vehicle architecture across multiple hardware platforms, multiple vehicles, and customer-facing scenarios. The integrated platform showed how cloud connectivity, graphical user interaction, application execution, and vehicle interfacing could be distributed across separate computing environments while still functioning as a coherent system. It also demonstrated that one board could be adapted for use in different vehicles with minimal software changes, with the main vehicle-specific software adjustments concentrated in the Dragoon ECU. After the demonstration approach was adapted for GM, the Lego Raptor and RC car versions made the showcase more portable and easier to deploy without relying on full vehicles. This expanded the usefulness of the demonstrator beyond the original full-vehicle setups and helped validate the technical feasibility of the architecture for broader customer-facing use cases.

b) In retrospect, what would you have done differently on this project?

In retrospect, I would have introduced a more service-oriented architecture earlier in the project. When the first showcases were prepared in July 2025, the turnaround was very fast and Eclipse eCAL was effective because the demonstration objective was relatively simple: move data through the platform and trigger visible behaviors such as blinking lights. As the showcase requirements evolved during the project, the demonstration moved beyond simple data transfer into higher-level vehicle features such as seat comfort scenarios, for example a relax mode that reclines the seat and sets a specific seat massage pattern. Because of that evolution, I would have moved the design toward Eclipse S-CORE so the demonstration could be restructured around service-oriented architecture principles rather than primarily data movement. That would have provided a stronger long-term foundation for future expansion of the platform.

End of Project Report 1. Continue to Project Report 2 on the next page.

Project Report 2

Completing Your Project Report

- Reports should be written in first-person narrative (I, my, me) to provide clear discussion and ownership of what you did versus what others did. Provide your own thoughts. Do not copy project documentation.
- You must provide clear citation and referencing when drawing on the work of others. Other people's original ideas and methods should be clearly distinguished, and other people's words, illustrations and diagrams should be clearly indicated regardless of whether they are copied exactly, paraphrased, or adapted.
- Diagrams from the project documentation may be helpful, but the text should be in your own words. Please ensure that diagrams are relevant, readable, and help the assessor to understand what you did as a member of the project team.
- If sections or components of the Project Report (below) are not relevant to your participation in the project, then leave the section blank.

Focus on quality, rather than quantity. Each Project Report should be no more than 4-5 pages.

1. About Project 2

Project 2 Name	InfraDID Config			
Nature of Project	Internal diagnostic automation and base software configuration tool development			
Location/s of Project	Auburn Hills, Michigan, USA			
Name of Your Employer	BorgWarner, Inc.			
Total Project Duration	From	02/22	To	04/22
Your Involvement^	From	02/22	To	04/22

^This project must have been undertaken within the last four years.

1.1 Client Details

Client's Business Name	BorgWarner, Inc.	
Business Address	3800 Automation Ave #100 Auburn Hills, Michigan 48326 USA	
Web Address	www.borgwarner.com	
General Email Address	info@borgwarner.com	
Contact Name/s	Ben Griffiths	
Contact Phone Number/s	+1 (248) 754 - 9600	

1.2 Project Resources

Your Team Size	1

Total Project Team Size	1

1.3 Personal Involvement

a) List the project phases you were involved in.

Start	End	Phase Description
02/22	02/22	Requirements analysis and problem definition
02/22	03/22	Tool architecture, parser design, and implementation
03/22	04/22	Output generation, integration support and rollout

b) Describe your role/s and responsibilities in the project.

I was the sole software designer and developer for this internal BorgWarner project. I defined the tool concept, analyzed the AUTOSAR diagnostic configuration inputs, designed the parsing and generation flow, implemented the tool in Python, and validated the generated outputs for integration into the base software. I coordinated with system engineers, software architects, managers, supervisors, and other software engineers to confirm the required infrastructure Diagnostic Data Identifiers (DIDs), expected data content, and memory placement constraints. These participants acted as stakeholders and reviewers, while I remained the only developer working on the project itself. My responsibilities included ensuring that the generated C source and linker configuration outputs were consistent with AUTOSAR diagnostic configuration and suitable for customer-deliverable base software.

2. Business Opportunity or Problem

a) Describe the business opportunity or problem(s) this project addressed.

The project addressed an internal software engineering inefficiency in the implementation of infrastructure-level Diagnostic Data Identifiers within AUTOSAR-based diagnostic software. Although the implementation logic for these DIDs was largely similar across programs, the actual data values and memory placement differed by project. Before this tool existed, there was no standardized implementation approach. Engineers had to manually review AUTOSAR ARXML client-server configuration, interpret data from the Complex Diagnostic Database (CDD), and implement the corresponding C code and linker configuration by hand. This process was slow, inconsistent, and prone to defects, especially when data needed to be placed correctly in ROM or Flash memory regions. The lack of automation increased turnaround time and created rework during integration and testing.

3. Solution

a) Discuss your contribution to the solution, project, or engagement.

I designed and implemented a Python-based automation tool called InfaDID Config. The tool parsed AUTOSAR ARXML configuration files, identified the client-server definitions associated with infrastructure DIDs, and cross-referenced the relevant diagnostic data definitions from the CDD file. Based on that information, it automatically generated the required C implementation and linker file configuration needed to integrate the DIDs into

the base software platform. I used the Mako templating engine to define reusable interface templates for each DID type supported by a given project or program, which allowed the generated outputs to remain standardized while still accommodating project-specific data and memory-location parameters. This reduced manual interpretation effort, improved consistency across programs, and made the implementation process more repeatable and less error-prone.

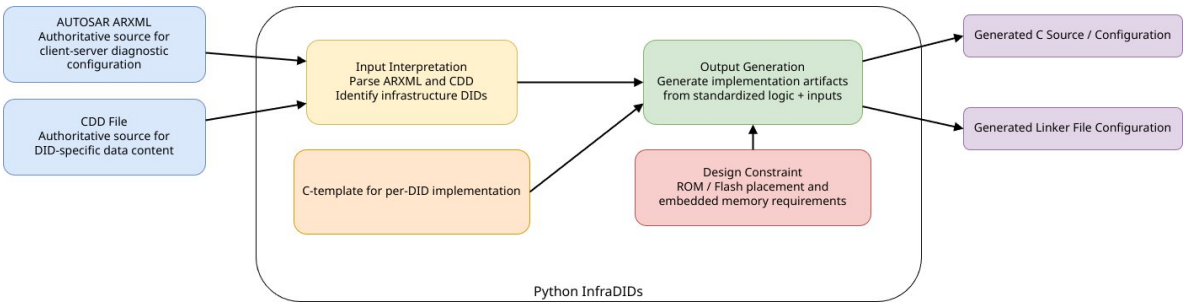
b) List the major deliverables of the project that you were responsible for or contributed to.

- Python automation tool for infrastructure DID configuration
  - ARXML parsing logic for AUTOSAR diagnostic definitions
  - CDD parsing logic for diagnostic data extraction
  - Mako-based templates for DID interface generation across supported projects/programs
- Generated C source/configuration outputs for infrastructure DIDs
- Generated linker file configuration for ROM/Flash placement

c) Describe any design or problem-solving methods you used during this project.

I approached the work by first analyzing the recurring implementation pattern used for infrastructure DIDs across projects. I separated the problem into three core parts: input interpretation, rules mapping, and output generation. I treated the ARXML file as the authoritative source for the diagnostic client-server configuration and the CDD file as the authoritative source for the DID-specific data content. I then designed the tool so that shared implementation logic would remain constant while configurable inputs would drive the generated output. To support this, I used the Mako templating engine to create reusable templates for DID interfaces, allowing project- or program-specific variations to be applied without rewriting the underlying generation logic. This reduced duplication and improved maintainability. I also considered embedded memory constraints as part of the design, since certain data had to be placed correctly in ROM or Flash. I validated the generated outputs against expected integration behavior and adjusted parsing and generation logic where inconsistencies were identified.

The diagram below shows the high-level design flow I implemented for the InfaDID Config tool, from diagnostic configuration inputs through automated generation of C and linker configuration outputs.



d) Was your solution/s implemented? If so, describe the role you had, if any, in the implementation.

Yes. My solution was implemented and became part of the internal software development workflow. I was responsible for the complete implementation of the tool, initial validation, and support for integrating its outputs into the base software. The generated C and linker

configuration artifacts were used as part of the software delivered to customers.

4. Results

a) Assess the overall success or failure of the project.

The project was successful. During my involvement between February 7, 2022 and April 21, 2022, implementation and testing for approximately 20 to 30 infrastructure DIDs typically required 1 to 2 weeks of engineering effort when performed manually. With the automation tool in place, the same work could be completed in about 3 days including testing. The main remaining effort shifted from manual implementation to clarifying which infrastructure DIDs were required and determining where the associated data should be stored in memory. The tool improved consistency, reduced manual coding errors, and provided a standardized method for implementing infrastructure DIDs across multiple programs.

b) In retrospect, what would you have done differently on this project?

In retrospect, I would have extended the tool to automatically generate test cases for each supported DID. That would have further reduced manual validation effort and improved confidence in the generated outputs during integration and regression testing.